

6.2. Sequential Search

6.2.1. Langkah-langkah sequential search

1. Membuat method sequential searching di mahasiswaBerprestasi23:

```
int sequentialSearching(double cari) {  
    int posisi = -1;  
    for (int i = 0; i < idx; i++) {  
        if (listMhs[i] != null &&  
Double.compare(listMhs[i].ipk, cari) == 0) {  
            posisi = i;  
            break;  
        }  
    }  
    return posisi;  
}
```

2. Membuat method tampil posisi di mahasiswaBerprestasi23:

```
void tampilPosisi(double x, int pos) {  
    if (pos != -1) {  
        System.out.println("Data : " + x + " ditemukan pada  
indeks " + pos);  
    } else {  
        System.out.println("Data : " + x + " tidak  
ditemukan");  
    }  
}
```

3. di class mahasiswaBerprestasi23 membuat method tampil data search:

```
void tampilDataSearch(double x, int pos) {  
    if (pos != -1) {  
        System.out.println("Data mahasiswa dengan IPK : " +  
x);  
        listMhs[pos].tampilInformasi();  
    } else {  
        System.out.println("Data mahasiswa dengan IPK : " + x  
+ " tidak ditemukan");  
    }  
}
```

4. Di class MahasiswaDemo23:

```
System.out.println("Data mahasiswa:");  
daftar.tampil();  
  
System.out.print("Masukkan IPK yang dicari (Sequential  
Search): ");  
double cari = readDoubleFlexible(sc);
```

```
int posisi = daftar.sequentialSearch(cari);  
daftar.tampilPosisi(cari, posisi);  
daftar.tampilDataSearch(cari, posisi);
```

6.2.2. Verifikasi hasil:

```
IPK : 3.42  
  
NIM : A12  
Nama : Budi  
Kelas : TI-1A  
IPK : 3.87  
  
NIM : A13  
Nama : Citra  
Kelas : TI-1A  
IPK : 3.12  
  
NIM : A14  
Nama : Dina  
Kelas : TI-1A  
IPK : 3.65  
  
NIM : A15  
Nama : Eka  
Kelas : TI-1A  
IPK : 3.25
```

```
Masukkan IPK yang dicari (Sequential Search): 3.12  
Data : 3.12 ditemukan pada indeks 2  
Data mahasiswa dengan IPK : 3.12  
NIM : A13  
Nama : Citra  
Kelas : TI-1A  
IPK : 3.12
```

6.2.3 Pertanyaan:

1. tampilDataSearch untuk menampilkan informasi dari seseorang yang kita cari.
tampilPosisi untuk menampilkan indeks ke berapa datanya berada.
2. berfungsi untuk menghentikan perulangan saat data sudah ditemukan, supaya pencarian tidak lanjut ke elemen berikutnya (lebih efisien dan memastikan yang diambil adalah temuan pertama).
3. Variabel pos menyimpan indeks hasil pencarian. Dipakai untuk:
 - a. mengecek ketemu/tidak (mis. pos == -1 berarti tidak ketemu),
 - b. mengambil data yang ditemukan (mis. listMhs[pos]).
4. Kalau ada lebih dari satu data dengan nilai sama, sequential search (scan dari awal) + ada break akan menampilkan data yang pertama kali ditemukan (indeks paling kecil).
5. Jika break dihapus, perulangan tetap lanjut sampai akhir; akibatnya pos akan berisi indeks data yang terakhir cocok (bukan yang pertama), dan prosesnya jadi lebih lama.

6.3 Binary Search

6.3.1 Langkah-langkah Binary search

1. masukan method findBinarySearch di MahasiswaBerprestasi:

```
int findBinarySearch(double cari, int left, int right) {
    if (left > right) {
        return -1;
    }

    int mid = (left + right) / 2;
    if (listMhs[mid] == null) {
        return -1;
    }

    int cmp = Double.compare(listMhs[mid].ipk, cari);
    if (cmp == 0) {
        return mid;
    }

    boolean ascending = listMhs[left] != null &&
listMhs[right] != null && listMhs[left].ipk <=
listMhs[right].ipk;

    if (ascending) {
        if (cmp > 0) {
            return findBinarySearch(cari, left, mid - 1);
        }
        return findBinarySearch(cari, mid + 1, right);
    }

    // Descending
    if (cmp < 0) {
        return findBinarySearch(cari, left, mid - 1);
    }
    return findBinarySearch(cari, mid + 1, right);
}
```

2. panggil method findBinarySearch:

```
System.out.print("Masukkan IPK yang dicari (Binary
Search): ");

double cari2 = readDoubleFlexible(sc);

int posisi2 = daftar.findBinarySearch(cari2);
daftar.tampilPosisi(cari2, posisi2);
```

```
daftar.tampilDataSearch(cari2, posisi2);
```

6.3.2 verifikasi hasil:

```
Masukkan IPK yang dicari (Binary Search): 3.25
Data : 3.25 ditemukan pada indeks 1
Data mahasiswa dengan IPK : 3.25
NIM   : A15
Nama  : Eka
Kelas : TI-1A
IPK   : 3.25
```

6.3.3 Pertanyaan:

1. Divide:

```
int mid = (left + right) / 2;
if (listMhs[mid] == null) {
    return -1;
}

int cmp = Double.compare(listMhs[mid].ipk, cari);
if (cmp == 0) {
    return mid;
}

boolean ascending = listMhs[left] != null &&
listMhs[right] != null && listMhs[left].ipk <=
listMhs[right].ipk;
```

2. Conquer:

```
if (ascending) {
    if (cmp > 0) {
        return findBinarySearch(cari, left, mid - 1);
    }
    return findBinarySearch(cari, mid + 1, right);
}

// Descending
if (cmp < 0) {
    return findBinarySearch(cari, left, mid - 1);
}
return findBinarySearch(cari, mid + 1, right);
```

3. Tetap, karna program sebenarnya tidak memerlukan data yang urut tapi data urut digunakan agar hasil pencarian benar kalau cuman berjalan ya bisa tnpa data yang urut.
4. Hasil pertama: 3.5 dgn 3.2? kurang dari.
Hasil kedua: 3.4 dgn 3.2? kurang dari.
Hasil ketiga: 3.2

Hasilnya sudah sesuai.

5. jika:

```
if (left > right) {  
    return -1;  
}
```

6. kode baru:

7.